A1.3 Operating systems and control systems

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- A1.3.1 Describe the role of operating systems
- A1.3.2 Describe the functions of an operating system
- A1.3.3 Compare different approaches for scheduling
- A1.3.4 Evaluate the use of interrupt handling and polling
- A1.3.5 Explain the role of the operating system in managing multitasking and resource allocation (HL)
- A1.3.6 Describe the use of the control system components (HL)
- A1.3.7 Explain the use of control systems in a range of real-world applications (HL)

A1.3.1 The role of operating systems

An operating system (OS) is the fundamental software that manages computer hardware and software resources and provides common services for computer programs. It acts as an intermediary between the user and the computer hardware, ensuring efficient and secure operation of the system.

Operating systems simplify user interactions with computer hardware by abstracting the underlying complexities. This means users and applications do not need to understand the detailed workings of hardware components such as CPUs, memory and input / output devices. Instead, the OS provides a set of high-level services and interfaces that hide these complexities, making the system easier to use and program.

The primary role of an operating system is to manage the computer's resources effectively. This includes:

- **CPU management:** allocating CPU time to various processes and ensuring efficient execution
- **memory management:** handling the allocation and de-allocation of memory to applications, and managing virtual memory to extend physical memory capacity
- **storage management:** organizing and managing data on storage devices, ensuring reliable data storage and retrieval
- **device management:** controlling and co-ordinating hardware devices, providing drivers and interfaces for seamless operation.

Some of the most famous modern operating systems are Microsoft Windows, Apple macOS and Linux on larger computers and laptops, with Android and iOS being more popular for smaller portable devices such as smartphones.

A1.3.2 Functions of an operating system

Operating system functions are multifaceted, ensuring that the system runs smoothly, efficiently and securely. Here, we will delve into the various critical functions of an operating system, illustrating how it maintains system integrity while running background operations and managing resources.

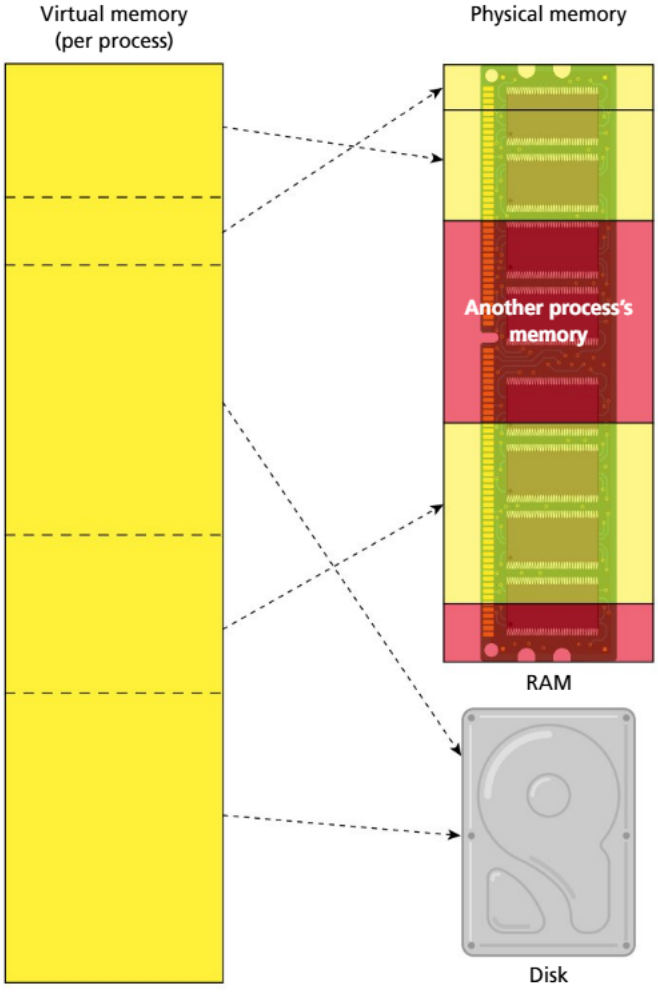
Memory management

◆ **Virtual memory:** a memory-management technique that allows a computer to use more memory than is physically available by temporarily transferring data from RAM to disk storage, enabling the execution of larger programs and multitasking.

Memory management is a fundamental function of an operating system (OS), involving the control and co-ordination of a computer's primary memory (RAM). The OS ensures that memory is allocated efficiently to processes and applications, maintains system stability and protects memory areas from unauthorized access.

When you open / run an application, the OS will load it into RAM. First, it will locate the application on the storage device (likely a hard disk drive or solid state drive) and read the executable file (.exe on Windows or .app on Mac). This file contains all the application's code and initial data.

The OS then allocates the necessary memory space for the application. This includes space for the code, data and any other required resources. The OS ensures that the application has enough space to execute without interfering with other processes.



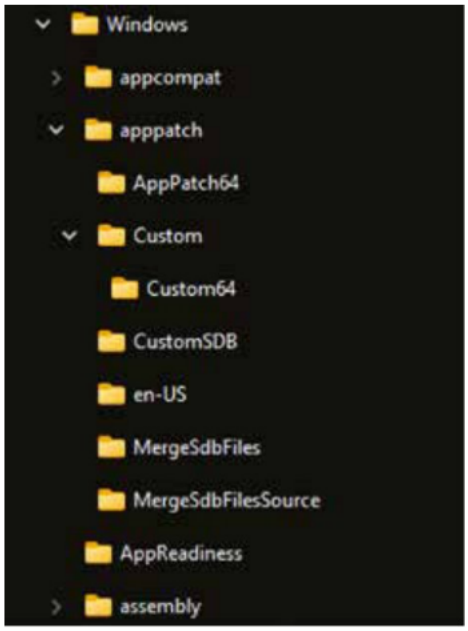
Relationship between virtual memory and physical memory

Once the application is loaded into RAM, the OS continuously manages memory to ensure efficient operation and system stability. While the application is running, the OS can dynamically allocate and de-allocate memory as needed by the application. The OS also ensures that each process operates within its own memory space. This prevents processes from interfering with each other, which enhances the security and stability of the system.

If the applications require more memory than is available in the RAM, the OS can use **virtual memory** to keep the system running smoothly. Virtual memory is when the OS allocates some space on the hard disk drive (HDD) or solid state drive (SSD) to use as RAM. This is not an ideal situation, as reading from a storage device is much slower than accessing RAM but, by switching processes with a higher priority into RAM, and those with lower priority into virtual memory, the OS is able to continue to run the system at an optimal level.

Top tip!

Think of memory management as organizing books on a bookshelf. The operating system allocates each book (process) its own space on the shelf (memory). Just as you wouldn't let books overlap or mix up, the OS ensures that each process has its own protected area in memory.



■ Windows File Explorer view displaying a hierarchical directory structure

File management

File management is a crucial function of an OS, involving the storage, retrieval, organization and manipulation of data on storage devices. The OS provides a structured way to manage files and directories, ensuring system stability and security.

The OS employs a hierarchical file system to organize files in a logical and structured manner. Files are organized into directories (or folders), which can contain further directories, creating a tree-like structure. This organization makes it easier for users and applications to locate and manage data.

The OS allows a number of set file operations that users and applications can perform on the files and directories, including creating new files and directories; reading and writing data to files; deleting files; renaming files; and moving or copying files and directories to different locations.

To ensure consistency and avoid conflicts, the OS enforces rules for naming files, ensuring no two files in the same directory have the same name. Files often have **extensions**, like .jpg or .exe, to indicate the file type and associated application although, depending on the OS settings, these may be hidden from the user.

■ MacOS finder view displaying a hierarchical directory structure

■ Some of the most commonly used file extensions

◆ **File extension:** a suffix at the end of a filename that indicates the file type and the program associated with opening or processing that file (e.g. .docx for Word documents, .jpg for images).

◆ **Defragmentation:** the process of reorganizing the data on a hard drive so that files are stored in contiguous blocks, reducing fragmentation and improving access speed and overall system performance.

When managing the storage of files, the OS uses various methods to improve performance; this includes how it allocates files on the physical storage medium, manages free space and performs maintenance on the saved data. One important maintenance task is **defragmentation**, which reorganizes fragmented data on a hard disk drive (HDD). Fragmentation occurs over time as files are created, modified and deleted, causing them to be scattered across different sectors of the disk. Defragmentation aims to rearrange these file fragments into contiguous sequences, so improving system performance.

◆ **Device drivers:** specialized software programs that allow the operating system to communicate with and control hardware devices, e.g. printers, graphics cards or network adapters, by providing the necessary instructions and protocols.

◆ **Buffering:** the process of temporarily storing data in a memory area (buffer) while it is being transferred between two devices or processes, helping to manage differences in data-flow rates and ensuring smooth, uninterrupted operation.

◆ **Caching:** the process of temporarily storing frequently accessed data in a high-speed storage area (cache) to reduce access time and improve system performance by enabling quicker retrieval of the data.

◆ **Spooling:** the process of queuing data or tasks in a buffer, typically for input / output devices such as printers, so that they can be processed sequentially and at their own pace, allowing the system to continue working on other tasks in the meantime.

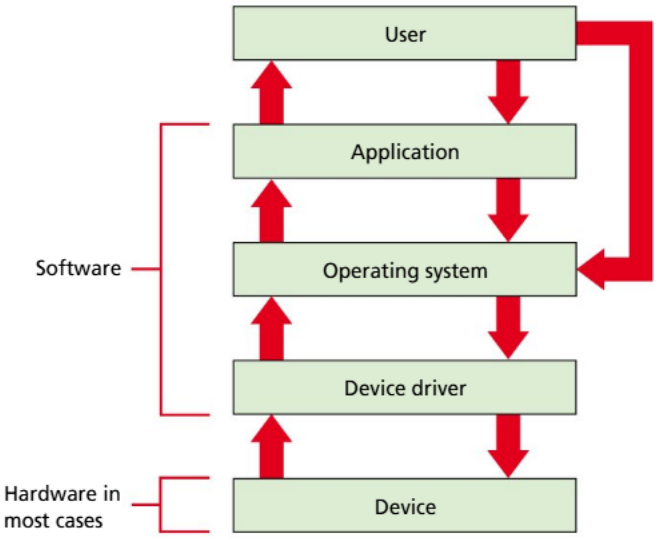
◆ **Plug and Play (PnP):** a technology that allows the operating system to detect, configure and install drivers automatically for new hardware devices when they are connected to the computer, enabling them to work without requiring manual set-up by the user.

Device management

Operating system device management ensures that hardware devices operate efficiently and interact seamlessly with software applications. The OS controls and co-ordinates the use of hardware components such as printers, disk drives, display screens, keyboards and network interfaces through specialized software called **device drivers**. Device drivers are essential programs that enable the OS to communicate with connected hardware. Each piece of hardware requires a specific driver to function correctly and efficiently. The OS uses these drivers to provide a uniform interface, allowing applications to interact with the hardware without needing to understand the hardware's specifics.

The OS also handles input/output (I/O) management to co-ordinate data transfer between the computer and peripheral devices. It employs techniques such as **buffering**, **caching** and **spooling** to optimize performance and reliability. Buffering involves using temporary storage areas to hold data while it is being transferred between devices, accommodating speed differences between the CPU and peripheral devices. Caching stores frequently accessed data to reduce access times, improving overall system efficiency. Spooling queues data and sends it in a manageable order to devices such as printers that cannot handle interleaved data streams.

Modern operating systems have introduced user-friendly ways of connecting devices, such as **Plug and Play (PnP)** technology. With PnP, the OS can automatically identify a device that has been attached and install the necessary drivers and configure settings without the need for user interaction. The OS is also capable of detecting errors and taking action to recover from them, by resetting the device, reinitializing drivers or notifying the user of the issue. Additionally, the OS provides security and access control to ensure that only authorized users and processes can access certain devices, maintaining system integrity and security.



Device drivers act as intermediaries between the operating system and hardware devices, enabling software to communicate effectively with hardware components

Scheduling

The OS schedules the process of managing the execution of multiple processes by determining which process runs at any given time. The scheduler is responsible for allocating CPU time to processes, ensuring efficient and fair use of system resources. This function is crucial for multitasking environments, where multiple applications and background processes run concurrently. By implementing various scheduling algorithms, the OS aims to optimize performance, reduce wait times and maintain system stability.

In Section A1.3.3, we will delve deeper into specific scheduling approaches, including first come first served, round robin, multilevel queue scheduling and priority scheduling. Each method has its own advantages and trade-offs, and its suitability depends on the specific requirements and goals of the system.

Security

The OS security is designed to protect the integrity, confidentiality and availability of information and resources within the computer system. It has mechanisms to safeguard against threats, prevent unauthorized access and ensure that users and applications can operate securely.

User authentication

User authentication can be used in multiple ways. Initially, the OS may require the user to authenticate themselves to log in to the system. It would require the user to provide credentials such as a username and password, biometric data or **security tokens**. Once logged on, based on the user settings, the OS can then grant or prevent access to certain files, folders or applications on the system. This allows systems to have multiple users with different access credentials, such as an administrator and a standard user.

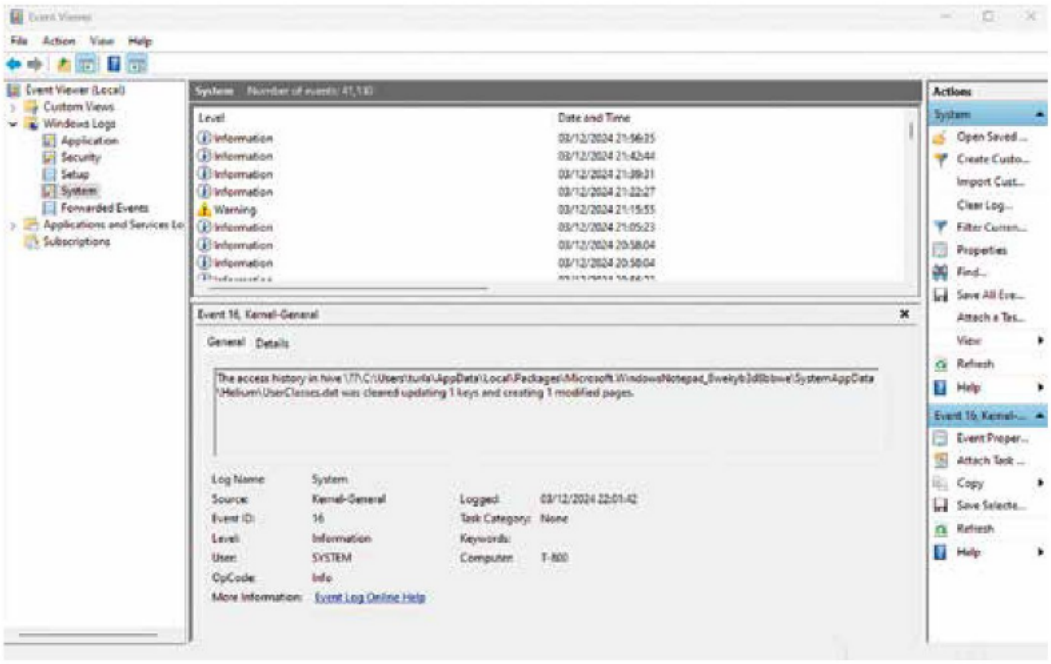
◆ **Security tokens:** physical or digital devices that generate or store authentication credentials, such as one-time passwords or cryptographic keys, used to verify a user's identity and secure access to systems, networks or online services.

Encryption

The OS can provide tools and frameworks for encrypting files, communication channels and devices. The encryption ensures that the data, even if it is intercepted or accessed by unauthorized individuals, cannot be read without the decryption key.

Auditing and monitoring

All system activities, such as users logging in, file access, system errors and administrator actions, are tracked by the OS in a log. These logs are used for auditing and monitoring purposes and can help administrators detect suspicious activities and potential security breaches. They can also be used to help identify issues and areas for improving the system performance.



Windows Event Viewer

Malware protection

The OS includes mechanisms to detect and prevent **malware** infections and intruders. These include antivirus software, firewalls and intrusion detection systems (IDS). These tools scan for malicious software, monitor network traffic and block unauthorized access attempts, protecting the system from **viruses**, **worms**, **trojans** and other malicious threats.

◆ **Malware:** a general term for any software designed with malicious intent, e.g. viruses, worms, trojans, spyware and ransomware, which can damage systems, steal data or disrupt operations.

◆ **Viruses:** malicious software programs that attach themselves to legitimate files or programs and spread to other files or systems, often causing damage or disruption.

◆ **Worms:** self-replicating malware that spreads across networks without needing to attach to other programs, exploiting vulnerabilities to infect multiple systems.

◆ **Trojans:** deceptive programs that appear legitimate but carry hidden malicious code, which can create backdoors, steal data or cause harm once executed by the user.

Accounting

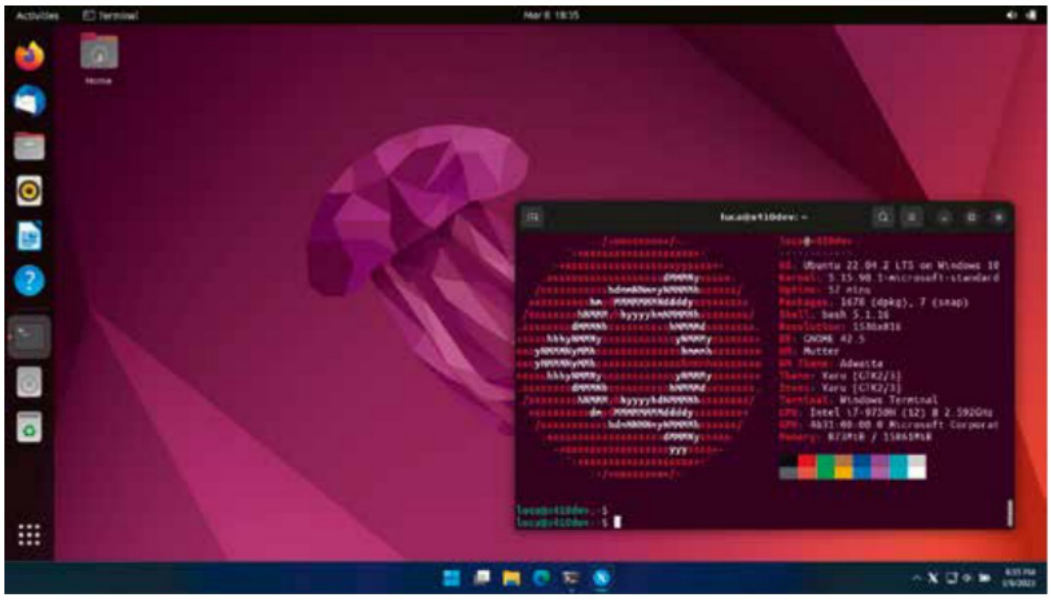
Operating system accounting functions are essential for monitoring and managing the usage of system resources. These functions track resource consumption by users and processes, providing valuable data for system administrators to analyse performance, allocate costs and optimize resource utilization. The key aspects of OS accounting functions include:

- **Resource usage tracking**
  The OS continuously monitors the consumption of various resources by users and processes; this includes tracking:
  - **CPU usage:** the amount of CPU time consumed by each process or user
  - **memory usage:** the amount of RAM allocated and used by each process
  - **disk usage:** the amount of storage space occupied by files and directories owned by each user
  - **network usage:** the volume of data sent and received over the network by each process or user.
- **Process accounting**
  Process accounting involves maintaining detailed records of each process that runs on the system; this includes information such as:
  - **process ID:** a unique identifier for each process
  - **user ID:** the identifier of the user who initiated the process
  - **execution time:** the total CPU time used by the process
  - **start and end times:** the timestamps indicating when the process started and finished
  - **resource consumption:** details on the amount of memory, disk I/O and other resources used by the process.
- **User accounting**
  User accounting tracks the resource usage by individual users or user groups; this information is crucial for:
  - **cost allocation:** in multi-user environments, such as universities or enterprises, resource usage data can be used to allocate costs to different departments or projects based on their consumption
  - **quota management:** enforcing resource usage limits for users to prevent any single user from monopolizing system resources; this can include disk quotas, limiting the amount of storage a user can use and memory quotas.
- **Performance monitoring**
  The OS accounting functions are integral to performance monitoring; by analysing resource usage data, system administrators can identify:
  - **bottlenecks:** areas where resources are being overutilized, causing performance degradation
  - **underutilization:** resources that are underused, indicating potential areas for optimization
  - **trends:** patterns in resource usage over time, which can inform capacity planning and system upgrades.

- **Auditing and reporting**
  The OS generates detailed reports based on the collected accounting data; these reports can be used for:
  - **auditing:** ensuring compliance with organizational policies and regulatory requirements by reviewing resource usage and access patterns
  - **security analysis:** detecting unusual or suspicious activity by analysing resource usage anomalies
  - **resource management:** making informed decisions about resource allocation, system configuration and future investments.
- **Billing and chargeback**
  In environments where resource usage needs to be billed to individual users or departments, such as cloud-computing services or academic institutions, OS accounting functions enable:
  - **usage-based billing:** charging users based on their actual resource consumption, such as CPU hours, memory usage and network bandwidth
  - **chargeback:** allocating costs to different departments or projects based on their resource usage, promoting accountability and efficient resource use.

Graphical user interface

By offering visual elements such as windows, icons, menus and pointers, the OS provides a user-friendly environment for interacting with the computer and allows the user to execute commands, manage files and run applications.



The Ubuntu GUI

User interface elements

The user interface elements provided by the OS allow the user to interact with the system intuitively. These elements include windows, which display the contents of applications, documents or system information. Icons give graphical representation to applications, files and system functions, providing quick access to frequently used items. Menus offer lists of commands or options to access various functions, making navigation and operation straightforward. Pointers, usually represented by an arrow or cursor, can be controlled with an input device like a mouse, enabling users to select, drag and interact with GUI elements seamlessly.

Application management

The GUI facilitates the management of and interaction with multiple applications, enhancing user productivity and experience. Task switching allows users to move quickly between open applications, with features like the taskbar or application switcher (Alt + tab) enabling efficient navigation. Window management helps users organize their workspace by arranging, overlapping and tiling windows. Features such as snapping windows to edges or corners create a split-screen effect for multitasking. The desktop environment, where users can place icons, shortcuts and widgets, allows for a personalized and organized workspace, catering to individual preferences.

File and system management

The GUI simplifies file and system management tasks through visual tools and interfaces. A GUI-based file management tool, like the File Explorer, allows users to navigate directories, view file properties and perform operations such as copying, moving, deleting and renaming files and folders. This tool often includes features such as search, sort and filter to facilitate file management. The GUI also provides access to system settings and control panels, enabling users to configure hardware, software and system preferences through intuitive graphical interfaces. Drag-and-drop functionality offers a user-friendly method for transferring files and data between applications and directories.

Accessibility

The GUI includes features that enhance accessibility for users with disabilities, ensuring that the system is usable by everyone. Screen readers convert text and GUI elements into speech or Braille, helping visually impaired users to navigate the system. High-contrast themes and screen magnifiers improve readability for users with low vision. Keyboard shortcuts allow users to perform actions quickly without relying on a mouse or touchpad, benefiting users with limited mobility.

■ MacOS accessibility features

Visual feedback

The GUI provides immediate visual feedback to user actions, enhancing the overall user experience. Progress indicators, including progress bars and loading animations, inform users about the status of ongoing operations such as file transfers or software installations. Notifications, in the form of pop-up messages and alerts, keep users informed about important events, updates or errors. Tooltips – small informative text boxes that appear when users hover over icons or interface elements – provide additional information or guidance.

Customization and personalization

The GUI allows users to personalize their computing environment to suit their preferences, making the system more enjoyable and efficient to use. Users can change the appearance of the GUI by selecting different themes, wallpapers and window styles, creating a more customized experience. Widgets and gadgets – small applications that provide quick access to information and tools such as weather updates, calendars and system monitors – enhance the functionality and aesthetics of the desktop. These customization options enable users to tailor their workspace to their needs and preferences, improving overall satisfaction and productivity.

Virtualization

Virtualization is a key feature of modern operating systems that allows multiple virtual machines (VMs) to run on a single physical machine. Each VM operates independently, with its own OS and applications. The operating system manages this through a **hypervisor**, which allocates CPU time, memory and storage to each VM, ensuring efficient use of resources and allowing each VM to function as if it were on a separate physical machine.

Virtualization enhances security and stability by isolating VMs from each other, preventing issues in one VM from affecting others. It also allows for snapshots and backups, enabling administrators to save the current state of a VM and revert to it if necessary. Live migrations, which move VMs from one physical host to another without interrupting services, are another crucial feature, aiding in **load balancing** and hardware maintenance.

Additionally, virtualization supports disaster recovery and cloud services. By allowing VMs to be easily backed up and restored, the OS ensures that critical applications and data can be quickly recovered in case of failure. Virtualization also enables dynamic scaling of IT infrastructure in cloud environments, allowing businesses to adapt quickly to changing demands.

◆ **Hypervisor:** software that creates and manages virtual machines by allowing multiple operating systems to run simultaneously on a single physical machine, sharing the underlying hardware resources.

◆ **Load balancing:** the process of distributing network or application traffic across multiple servers or resources to ensure optimal performance, reliability and availability, preventing any single server from becoming overwhelmed.

Networking

Operating systems manage and facilitate network communication. One of the primary functions of an OS in networking is to establish and maintain network connections. The OS manages network interfaces and protocols, enabling devices to connect to local area networks (LANs), wide area networks (WANs) and the internet. It configures network settings, assigns IP addresses through DHCP (Dynamic Host Configuration Protocol) and handles the underlying hardware, such as network interface cards (NICs), ensuring that devices can communicate effectively.

The OS also provides essential services for data transmission and communication between devices. It implements various networking protocols, such as TCP/IP (Transmission Control Protocol/Internet Protocol), which govern how data is packetized, addressed, transmitted, routed and received. The OS ensures data integrity and efficient transmission by handling error checking, flow control and congestion avoidance. Additionally, the OS supports higher-level protocols and services, such as HTTP for web browsing, FTP for file transfers and

SMTP for email communication, facilitating seamless interaction between applications and network resources.

Security and access control are critical network functions managed by the OS. The OS employs firewalls, encryption and authentication mechanisms to protect data as it travels across networks. Firewalls monitor and control incoming and outgoing network traffic based on predetermined security rules, helping to prevent unauthorized access and attacks. The OS also supports virtual private networks (VPNs), which encrypt data and create secure connections over public networks, ensuring privacy and security for remote users. These security features safeguard network communication and ensure that only authorized users and devices can access network resources.

REVIEW QUESTIONS

1. What is the primary role of an operating system in a computer?
2. How does the operating system abstract hardware complexities for users? Provide an example.
3. Explain how the operating system manages multitasking and resource allocation. Why is this important?
4. What are some of the key challenges the operating system faces in resource management?
5. What is memory management, and how does the operating system ensure efficient use of memory?
6. Describe the role of device drivers in operating system device management.
7. How does the operating system manage files and directories? Give examples of file-management operations.
8. What is the importance of process scheduling, and what are some common scheduling algorithms used by operating systems?
9. Explain how the operating system handles security and access control. Why are these functions critical?

ACTIVITY

Risk-taker: Research and information literacy

In this task, you will improve your research and information literacy skills by installing Ubuntu, a popular Linux distribution, and comparing it with another operating system of your choice.

1. Install Ubuntu on to a device (this could be a spare PC, a Raspberry Pi, a virtual machine or even a USB drive). Document the installation process, noting how the OS manages hardware and interacts with the user.
2. Research and compare:
   - a. Choose another OS you are familiar with (for example Windows or macOS).
   - b. Research the difference in how Ubuntu and your chosen OS handle key aspects such as:
     - i. user interface (including accessibility features)
     - ii. file management
     - iii. memory management
     - iv. software installation and updates.
3. Present your findings:
   - a. Create a comparison table or chart to visually present the differences.
   - b. Include a reflection on what you learned.

PROGRAMMING EXERCISES

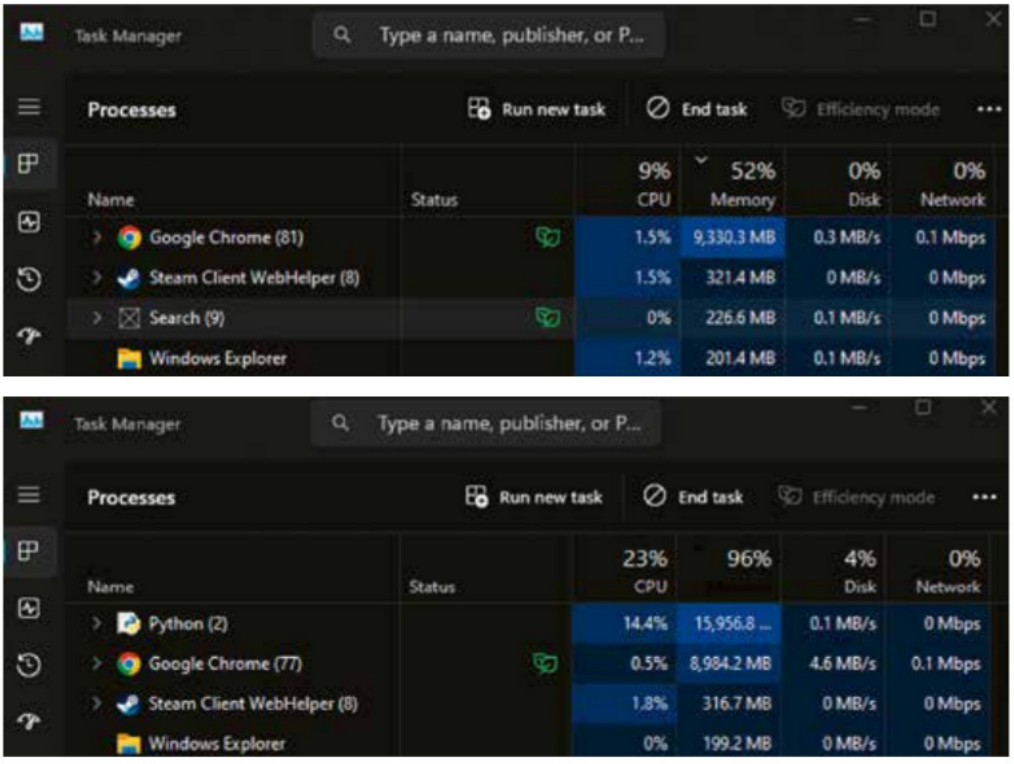
1 Run the simple Python "Memory Hog" program below, which continuously allocates memory until the system runs out.

Important note: Running this script may cause your system to become unresponsive, as it will use up all available memory. It's recommended to run this in a controlled environment, such as a virtual machine. You can use Ctrl + C (Windows) or Cmd + C (Mac) to stop the program running.

Python

```
memory_hog = []
try:
    while True:
        # Allocate a large list and append it to the
        # memory_hog list
        memory_hog.append([0] * 10**6)  # Each list has
        # 1 million zeros
        print(f"Allocated {len(memory_hog)} million items")
except MemoryError:
    print("Memory allocation failed! The system has run out
    of memory.")
```

This script creates a large list and appends a million zeros each time it loops. When the system can no longer allocate memory (due to running out of available RAM), a "MemoryError" exception is raised and will be output. You can monitor your system memory using Task Manager on Windows or Activity Monitor on macOS.



■ Windows Task Manager showing the memory usage of the program above as it runs

2 **File-management tasks**

a Creating directories and files:

Step 1: Open the File Explorer (Windows) or Finder (macOS) on your computer.

Step 2: Navigate to a location where you can create a new directory (for example your desktop or documents folder).

- Windows: Right-click > New > Folder
- MacOS: Right-click or Ctrl-click > New Folder

Step 3: Inside "ProjectFiles", create three subfolders named "Reports", "Data" and "Images".

Step 4: Within the "Reports" folder, create three text files named "Report1.txt", "Report2.txt" and "Summary.txt". Use a text editor (for example Notepad or TextEdit) to create these files, and include a few lines of sample text in each file.

b Moving files:

Step 1: Move "Report2.txt" from the "Reports" folder to the "Data" folder.

- Windows: Drag and drop the file to the new location or right-click > Cut, then right-click in the destination folder and select Paste
- MacOS: Drag and drop the file or use Cmd + C (copy) and Cmd + V (paste).

Step 2: Move "Summary.txt" from the "Reports" folder to the "Images" folder using the same methods.

c Renaming files:

Step 1: Rename "Report1.txt" to "FinalReport.txt".

- Windows: Right-click the file > Rename
- MacOS: Click the file name once to select, then click again to edit.

Step 2: Rename the "Data" folder "ProjectData".

d Deleting files and directories:

Step 1: Delete the "Images" folder along with its contents.

- Windows: Right-click the folder > Delete
- MacOS: Drag the folder to the Trash or right-click > Move to Trash.

Step 2: Recover the deleted "Images" folder from the Recycle Bin (Windows) or Trash (macOS).

- Windows: Open the Recycle Bin, right-click on the "Images" folder, and choose "Restore"
- MacOS: Open the Trash, locate the "Images" folder, right-click it, and choose "Put Back" to restore it to its original location.

3 Research how to complete the same tasks away from the GUI using the OS terminal (Command Prompt / cmd on Windows or Terminal on macOS).

4 **Accessibility features**

Complete one of the following tasks on your chosen OS.

a Explore accessibility features on Windows:

Step 1: Open the Control Panel by pressing Windows + R, typing control, and pressing Enter.

Step 2: Navigate to Ease of Access Center.

Step 3: Explore various accessibility features, such as Narrator (screen reader), Magnifier, High Contrast Mode, Speech Recognition and On-Screen Keyboard.

Step 4: Activate and interact with at least two of these features, for example turn on Narrator and Magnifier, and navigate through the desktop or a web page to understand how these features assist users.

b Explore accessibility features on macOS:

Step 1: Open System Preferences by clicking on the Apple menu and selecting System Preferences.

Step 2: Go to the Accessibility section.

Step 3: Explore various features, such as VoiceOver (screen reader), Zoom, Display (for colour filters, invert colours), Speak Selection and Dictation.

Step 4: Activate and interact with at least two of these features, for example enable VoiceOver and Zoom and use them to navigate the system or read through a document.

◆ **Monopolize resources:** the control or domination of the use of system resources (e.g. CPU, memory or network bandwidth) by a single process or user, often to the detriment of other processes or users, leading to inefficiency or system slowdowns.

A1.3.3 Approaches for scheduling

An operating system needs scheduling methods to efficiently manage the execution of multiple processes, ensuring optimal use of the CPU and other resources. Scheduling determines the order in which processes are granted access to the CPU and their duration, balancing the needs of various applications and maintaining system responsiveness. Without effective scheduling, processes could experience significant delays or **monopolize resources**, leading to poor performance and user frustration. In the following section, we will examine several different scheduling methods, including first come first served, round robin, priority scheduling and multilevel queue scheduling, each with its own advantages and trade-offs.

First come first served

FCFS: waiting in line to be served

First come first served (FCFS) is one of the simplest scheduling algorithms used by operating systems to manage the execution of processes. In FCFS scheduling, the processes are executed in the order they arrive in the ready queue. When a process arrives, it is added to the end of the queue. The CPU scheduler selects the process at the front of the queue and assigns it to the CPU until it completes its execution or moves to an I/O wait state. Once the current process is finished, the next process in the queue is selected, and this continues until all processes have been executed.

The simplicity of FCFS makes it easy to implement and understand. However, it has some drawbacks, such as the "convoy effect", where short processes may be delayed by long-running processes, leading to increased waiting time and lower system throughput. This method is non-pre-emptive, meaning that once a process is assigned to the CPU, it cannot be interrupted until it completes, which can cause inefficiency in certain scenarios.

Advantages	Disadvantages
Simple and easy to implement	Convoy effect can cause significant delays
Fair, as it processes requests in the order they arrive	Non-pre-emptive nature can lead to inefficiency and longer than average waiting times

In this example, P1 arrives first and is executed immediately, followed by P2 and P3 in the order they arrive. Each process runs to completion before the next process begins.

Process	Queue
P1 arrives	P1
P2 arrives	P1, P2
P3 arrives	P1, P2, P3

	Time									
Process	0	1	2	3	4	5	6	7	8	9
P1										
P2										
P3										

Round robin: each task takes a turn in a continuous cycle, ensuring that each one gets equal time

Round robin

Round robin (RR) is a pre-emptive scheduling algorithm designed to provide fair time-sharing among processes. Each process is assigned a fixed time slice, known as a "quantum", during which it can execute. When a process's quantum expires, the CPU scheduler pre-empts the process and places it at the end of the ready queue, then selects the next process in line for execution. This cycle continues until all processes are completed.

The primary advantage of round robin scheduling is that it ensures a high level of responsiveness, as no process can monopolize the CPU for an extended period. This approach is especially effective in time-sharing systems where multiple users or applications need to interact with the CPU frequently. The length of the time quantum is critical: if it is too short, the system spends too much time switching between processes; if it is too long, it resembles FCFS scheduling.

Advantages	Disadvantages
Fair allocation of CPU time between processes	Time quantum selection is crucial for performance
High responsiveness and improved system interactivity	High context-switching overhead if the time quantum is too short
Prevents any single process from monopolizing the CPU	Potential inefficiency if processes frequently complete their tasks within a single quantum

In this example, process P1 runs from time 0 to 2, then is pre-empted. Process P2 runs from time 2 to 4, then is pre-empted. Then, process P3 runs from time 4 to 6 and is pre-empted. The cycle repeats, with P1 running again from time 6 to 8, P2 from 8 to 10 and P3 from 10 to 12.

Process	Queue
P1 arrives	P1
P2 arrives	P1, P2
P3 arrives	P1, P2, P3

	Time											
Process	0	1	2	3	4	5	6	7	8	9	10	11
P1												
P2												
P3												

Priority scheduling

An example of a priority lane at an airport, where passengers with higher priority (such as first class ticketholders or those with a disability) are given faster service

Priority scheduling is a method where each process is assigned a priority level, and the CPU is allocated to the process with the highest priority. Processes with higher priority levels are executed before those with lower priority levels. If two processes have the same priority, they are scheduled according to their arrival order, typically using FCFS. Priority can be either static (meaning it is assigned when the process is created and does not change) or dynamic (meaning it can change over time based on various factors such as ageing).

The main goal of priority scheduling is to ensure that critical tasks are executed as soon as possible, enhancing the responsiveness of high-priority processes. However, a significant drawback is the potential for low-priority processes to suffer from starvation if high-priority processes continually dominate CPU time. To mitigate this, some systems implement ageing, which gradually increases the priority of waiting processes to ensure they eventually receive CPU time.

Advantages	Disadvantages
Prioritizes important tasks, improving system responsiveness for critical applications	Risk of starvation for low-priority processes
Flexible, as priorities can be adjusted dynamically based on system needs	More complex to implement and manage than simpler scheduling methods
	Priority inversion, where a lower-priority process holds a resource needed by a higher-priority process, can be an issue if not handled properly

In this example, P1 and P4, both high-priority processes, are executed first. After the high-priority processes are completed, the medium-priority process P2 is executed, followed by the low-priority process P3.

Process	Arrival time	Priority level
P1	0	High
P2	1	Medium
P3	2	Low
P4	3	High

	Time							
Process	0	1	2	3	4	5	6	7
P1 (High)								
P2 (Medium)								
P3 (Low)								
P4 (High)								

Multilevel queue scheduling

Multilevel queue scheduling is a scheduling algorithm that partitions the ready queue into several separate queues, each with its own scheduling algorithm and priority level. Processes are permanently assigned to one of these queues based on certain characteristics, such as process type, priority or memory requirements. Each queue may use a different scheduling algorithm, such as FCFS or round robin, and the queues themselves are scheduled in a specific order, often based on priority.

In this approach, higher-priority queues are given more CPU time compared to lower-priority queues. For example, interactive processes might be placed in a high-priority queue scheduled with round robin, while batch processes are placed in a lower-priority queue scheduled with FCFS. The CPU scheduler selects processes from the highest-priority queue first, moving to lower-priority queues only when the higher-priority queues are empty.

Advantages	Disadvantages
Flexibility in handling different types of processes	Starvation of lower-priority processes if higher-priority queues are frequently occupied
Prioritizes critical and interactive processes, improving responsiveness for important tasks	Complex implementation and management
Different scheduling algorithms can be tailored to the needs of each queue	Processes are permanently assigned to queues, which might not be optimal if their behaviour changes over time

In this example, processes in the high-priority queue (P1 and P2) are scheduled first using round robin. Both P1 and P2 complete. The scheduler then moves to the medium-priority queue (P3), and then to the low-priority queue (P4) only when the high-priority queue does not have processes ready to run. However, it frequently checks back to the high-priority queue, which is why P2 (high) runs again after P4 (low).

Queue priority	Process type	Scheduling algorithm	Process queue
Q1 – High	Interactive	Round robin (time quantum = 2)	P1, P2
Q2 – Medium	Batch	FCFS	P3
Q3 – Low	Background / idle	FCFS	P4

	Time												
Process	0	1	2	3	4	5	6	7	8	9	10	11	12
P1 (High)													
P2 (High)													
P3 (Medium)													
P4 (Low)													

Common mistake

Don't forget that context switching – when the CPU switches from one task to another – can slow things down. Even though it allows for multitasking, too much switching can reduce how efficiently the system runs because the CPU spends time saving and restoring tasks.

REVIEW QUESTIONS

1. Explain the difference between first come first served (FCFS) scheduling and round robin scheduling. How does the choice of time quantum in round robin affect process performance?
2. What is the purpose of priority scheduling, and how does it differ from multilevel queue scheduling? Provide an example of where each might be preferred.
3. In the context of operating systems, what is meant by context switching, and why is it important in process scheduling?
4. Describe how multilevel queue scheduling works and discuss its advantages and disadvantages.
5. How does the operating system ensure fairness in scheduling while also optimizing performance? Discuss this in the context of round robin and priority scheduling.
6. What are the potential drawbacks of using a first come first served scheduling algorithm in a multi-user environment?
7. Why might an operating system choose to implement a hybrid scheduling approach, and what benefits does this provide?

PROGRAMMING EXERCISE

Write a program to simulate different CPU scheduling algorithms and analyse their performance on process execution. Here is an example of how you could implement FCFS scheduling in Python.

Python

```
# Define a class to represent a process in the system
class Process:
    def __init__(self, pid, arrival_time, burst_time):
        self.pid = pid  # Process ID
        self.arrival_time = arrival_time  # The time at which the process
        # arrives in the system
        self.burst_time = burst_time  # The total time required by the
        # process to complete execution
        self.waiting_time = 0  # Time the process has to wait before it
        # starts execution
        self.turnaround_time = 0  # Total time taken from arrival to
        # completion (waiting_time + burst_time)
# Define a function to simulate FCFS scheduling
def calculate_fcfs(processes):
    start_time = 0  # Variable to track the current time at which a process
    # starts execution
    # Iterate over each process in the list
    for process in processes:
        # If the current time is less than the process's arrival time,
        # the CPU remains idle until the process arrives
        if start_time < process.arrival_time:
            start_time = process.arrival_time
        # Calculate the waiting time for the process
```

```
        process.waiting_time = start_time - process.arrival_time
        # Calculate the turnaround time for the process
        process.turnaround_time = process.waiting_time + process.burst_time
        # Update the current time to reflect the process's execution
        start_time += process.burst_time
    # After all processes are scheduled, calculate the total and average
    # waiting / turnaround times
    total_waiting_time = sum([p.waiting_time for p in processes])   # Sum of
    # all waiting times
    total_turnaround_time = sum([p.turnaround_time for p in processes]) # Sum
    # of all turnaround times
    avg_waiting_time = total_waiting_time / len(processes)   # Average waiting
    # time
    avg_turnaround_time = total_turnaround_time / len(processes) # Average
    # turnaround time
    # Display the results in a table format
    print("Process\tArrival Time\tBurst Time\tWaiting Time\tTurnaround Time")
    for process in processes:
        print(f"{process.pid}\t{process.arrival_time}\t\t{process.burst_time}\
        t\t{process.waiting_time}\t\t{process.turnaround_time}")
    # Print the calculated average times
    print(f"\nAverage Waiting Time: {avg_waiting_time:.2f}")
    print(f"Average Turnaround Time: {avg_turnaround_time:.2f}")
# Example process list to simulate FCFS scheduling
processes = [
    Process(1, 0, 5),   # Process 1 arrives at time 0 and requires 5 time
    # units to complete
    Process(2, 2, 3),   # Process 2 arrives at time 2 and requires 3 time
    # units to complete
    Process(3, 4, 1),   # Process 3 arrives at time 4 and requires 1 time unit
    # to complete
    Process(4, 6, 7)    # Process 4 arrives at time 6 and requires 7 time
    # units to complete
]
# Sort the processes by their arrival time before running the FCFS algorithm
processes.sort(key=lambda x: x.arrival_time)
# Run the FCFS scheduling simulation
calculate_fcfs(processes)
```

A1.3.4 Interrupt handling and polling

Interrupt handling and polling are two fundamental techniques used by operating systems to manage communication between the CPU and peripheral devices. Each method has its own advantages and drawbacks that can affect system performance and efficiency, depending on the context in which they are used.

Interrupt handling

Interrupt handling is a mechanism where peripheral devices signal the CPU to gain its attention and request service. When an event occurs, such as an input from a keyboard or data from a network interface, the device sends an interrupt signal to the CPU. The CPU then pauses its current operations, saves its state and executes an **interrupt service routine (ISR)** to address the event. This method allows the CPU to remain idle or perform other tasks until an event actually occurs, making it highly efficient in environments where events happen sporadically or unpredictably. Interrupt handling ensures that the CPU only deals with events when necessary, reducing unnecessary CPU cycles spent on checking for events.

- **Interrupt service routine (ISR):** a special function in a computer system that automatically executes in response to an interrupt signal, handling specific tasks, e.g. processing input from hardware devices or managing system events, before returning control to the main program.
- **Latency:** the delay between the initiation of an action and the corresponding response, often referring to the time it takes for data to travel from its source to its destination in a network or system.

However, the frequent occurrence of interrupts can introduce processing overheads due to the context switching involved. Each time an interrupt is handled, the CPU must save its current state and later restore it, which can be time-consuming and resource-intensive if interrupts are too frequent. Additionally, handling a high volume of interrupts can lead to increased power consumption, which is particularly critical for battery-powered devices. Despite these potential drawbacks, the efficiency of interrupt handling in managing sporadic events and minimizing CPU idle time makes it a preferred method in many real-time and interactive systems where immediate response to events is crucial.

Top tip!

An interrupt is like a doorbell ringing while you're busy working. You stop what you're doing (pause the current process), answer the door (handle the interrupt) and then return to your task. The operating system manages these interruptions efficiently so that your work (the main process) isn't significantly delayed.

Polling

Polling, on the other hand, involves the CPU periodically checking each peripheral device to see whether it requires attention. This method is straightforward and can be efficient in systems where events occur at regular, predictable intervals. Polling ensures controlled **latency**, as the CPU checks devices at predetermined times, making it suitable for real-time applications where timely response is crucial. Polling can be implemented easily and provides a simple mechanism to ensure that devices are checked regularly.

However, polling can lead to significant CPU processing overheads, as the CPU spends a considerable amount of time repeatedly checking devices instead of performing useful work. This continuous checking is resource-intensive and can detract from the system's overall efficiency. Additionally, polling is less power-efficient compared to interrupt handling, as the CPU remains active even when there are no events to process. This constant activity can drain the battery in portable devices more quickly than systems that utilize interrupt handling. In environments where event frequency is low or unpredictable, polling can be highly inefficient and wasteful, consuming CPU cycles without necessarily detecting any new events.

Interrupts vs polling

Criteria	Interrupts	Polling
Event frequency	Efficient for infrequent or unpredictable events, as the CPU only responds when an event occurs	More effective for regular, predictable events, as the CPU checks devices at set intervals regardless of event occurrence
CPU processing overheads	Lower overhead for infrequent events but can increase with high-frequency interrupts due to context switching	Higher overhead due to constant checking of devices, consuming CPU cycles even when no events occur
Power source	More power-efficient, especially for battery-powered devices, as the CPU remains idle until an event occurs	Less power-efficient, as the CPU remains active and continuously checks devices, leading to higher power consumption
Event predictability	Best for unpredictable events, as the system responds immediately to any event occurrence	Suitable for predictable events, ensuring regular checks at set intervals
Controlled latency	Can provide quick response times but, if interrupts are too frequent, it can lead to variability in response times	Provides controlled latency with predictable response times, as checks occur at regular intervals
Security concerns	Potentially more secure as the system can quickly respond to critical events, reducing the window for malicious activity	Less secure if polling intervals are too long, as it may delay the detection of critical events

The choice between interrupt handling and polling depends on the specific requirements and constraints of the system. Interrupt handling is generally more efficient for sporadic events and battery-powered devices, but can introduce overheads with high-frequency events. Polling offers predictable latency and is straightforward to implement, but can lead to inefficiencies and higher power consumption. Understanding the trade-offs between these methods is crucial for designing effective and efficient systems.

Common mistake

Remember that specific context is very important when deciding whether polling or interrupt handling is a better system solution. It is not a simple choice of one being better than the other. Interrupts are great for events that happen unpredictably, while polling is better for regular, predictable events. Make sure you understand the difference so you can choose the right method for each situation.

Real-world scenarios

Mouse and keyboard

When a user moves the mouse or presses a key, these devices generate interrupt signals that prompt the CPU to immediately pause its current tasks and execute the appropriate interrupt service routine (ISR). This ensures that user inputs are processed in real-time, providing instant feedback and seamless interaction. For example, as a user types, each keystroke generates an interrupt that the OS handles promptly, ensuring that characters appear on the screen without delay. Conversely, using polling for these devices would require the CPU to continuously check the status of the mouse and keyboard, leading to unnecessary processing overheads and increased power consumption, especially in battery-powered devices like laptops. Polling could also result in delayed responses if the CPU is busy with other tasks when a user input occurs.

For basic embedded systems like simple data-entry terminals or kiosks, where user interaction is infrequent and the system is primarily idle, polling might be sufficient. Polling at regular

intervals to check for user input can simplify the system design and avoid the overhead of setting up and handling interrupts. This is acceptable in low-activity environments where immediate response is not critical.

Network communications

When data packets arrive at a network interface card (NIC), they generate interrupt signals that alert the CPU to process the incoming data immediately. This prompt handling ensures that data is quickly received, processed and passed to the appropriate application, maintaining smooth and efficient network performance. For instance, during a video conference, interrupts enable real-time processing of audio and video data, ensuring minimal latency and high-quality communication. Conversely, using polling for network communications would require the CPU to continually check the NIC for new data, leading to increased processing overheads and potentially missing incoming packets if the CPU is occupied with other tasks. This could result in delays, reduced network performance and higher power consumption, especially in devices like smartphones or tablets.

In scenarios where network traffic is minimal and predictable, such as a remote monitoring system that periodically sends small data packets, polling can be more efficient. Polling at regular intervals to check for network activity reduces the complexity of interrupt handling and is sufficient to handle the infrequent, predictable communication needs.

Disk input / output operations

When a disk drive completes a read or write operation, it generates an interrupt signal that alerts the CPU to handle the data transfer immediately. This approach allows the CPU to execute other tasks while waiting for the disk operation to complete, enhancing overall system efficiency. For example, when a file is saved, the CPU can continue processing other applications until the disk signals that the write operation is finished, at which point the CPU promptly transfers the data to the appropriate location. Conversely, using polling for disk I/O operations would require the CPU to continuously check the status of the disk drive, leading to significant processing overheads and reduced efficiency. The CPU would waste valuable cycles repeatedly checking for completion, especially during lengthy disk operations, resulting in slower system performance and increased power consumption.

In a system where disk access is predictable and infrequent, such as a data logger that writes to a disk at fixed intervals, polling can be appropriate. Polling the disk for readiness before scheduled writes can simplify the implementation and eliminate the need for interrupt-driven complexity, making the system easier to manage.

Embedded systems

Embedded systems, such as those in automotive control units or industrial machinery, often need to respond quickly to sensor inputs and external signals. For example, in an automotive airbag system, sensors detecting a collision generate interrupt signals that prompt the CPU to immediately deploy the airbags. This rapid response is crucial for the safety and effectiveness of the system. Conversely, using polling in this scenario would require the CPU to continuously check sensor statuses, leading to increased processing overheads and potentially missing critical events if the CPU is occupied with other tasks. This delay in response could be catastrophic in time-sensitive applications.

In situations where events occur at regular, predictable intervals and the overhead of handling interrupts is not justified, polling can be a better approach. For example, in a climate-control system for a building, the temperature sensors might need to be checked at regular intervals to maintain a constant environment. Here, polling would be advantageous.

Real-time systems

In real-time systems, the choice between polling and interrupt handling depends on the specific requirements of the application. While interrupts are typically preferred for their quick response times, there are scenarios where polling can be more suitable.

For instance, in a real-time system that controls an industrial robot performing repetitive tasks at fixed intervals, polling can be more predictable and easier to manage. The robot might perform sensor checks and actuator adjustments at precise, regular intervals, ensuring that the tasks are executed in a controlled manner. This use of polling can simplify the design and avoid the overhead associated with frequent context switching that comes with interrupts, ensuring that the system meets its timing requirements consistently. In this scenario, the predictability and regularity of the events make polling a viable option, as it ensures that the system performs checks and adjustments at the exact required intervals without the complexity of handling numerous interrupts.

In a real-time system like an automotive airbag deployment system, interrupt handling is crucial. The system must respond immediately to sensor inputs indicating a collision. When sensors detect a rapid deceleration or impact, they generate interrupts that prompt the CPU to execute the airbag deployment routine instantly. This immediate response is essential to ensure the airbags deploy in time to protect the occupants. In such critical applications, the ability of interrupts to provide an immediate and high-priority response to specific events makes them the preferred choice, as any delay in processing could result in catastrophic consequences.

REVIEW QUESTIONS

1. Explain the fundamental difference between interrupt handling and polling in terms of how they manage CPU attention for peripheral devices.
2. In what scenarios might polling be more efficient than interrupt handling, and why?
3. Describe a situation in which interrupt handling could be preferred over polling, considering factors such as power consumption and response time.
4. How does the frequency of events affect the choice between interrupt handling and polling?
5. What are the potential drawbacks of using interrupt handling in a system with high event frequency?
6. Discuss how power source (battery vs mains power) can influence the choice between using interrupts or polling in a system.
7. How does the need for controlled latency impact the decision between using interrupt handling and polling? Provide an example of a system where controlled latency is critical.
8. Explain how security concerns could affect the choice between interrupt handling and polling in a networked system.

A1.3.5 The role of the operating system in managing multitasking and resource allocation (HL)

The operating system (OS) plays a critical role in managing multitasking and resource allocation, ensuring that multiple processes can run concurrently and efficiently on a computer system. Multitasking allows a system to perform multiple tasks seemingly simultaneously by quickly switching between them, while resource allocation ensures that each task receives the necessary resources (CPU time, memory, I/O devices) to execute properly. This involves several key functions and faces numerous challenges.

Task scheduling

Task scheduling is one of the primary responsibilities of the OS in a multitasking environment. The scheduler decides the order in which processes are executed, aiming to maximize CPU utilization and system responsiveness. As discussed in Section A1.3.3, there are various scheduling algorithms, such as first come first served (FCFS), round robin, priority scheduling and multilevel queue scheduling, each with advantages and drawbacks. The scheduler must balance the need to provide quick response times for interactive processes with the efficient processing of background tasks. This balancing act is crucial for maintaining system performance and user satisfaction.

Resource contention

Resource contention occurs when multiple processes compete for the same resources, such as CPU time, memory or I/O devices. The OS must manage this contention to prevent conflicts and ensure fair and efficient resource usage. Techniques like mutual exclusion are used to manage access to shared resources. Mutual exclusion is a key concept used in concurrent programming to prevent multiple processes from accessing a shared resource or critical section simultaneously. This ensures that only one process can use the resource at a time, preventing data corruption and ensuring consistency. Techniques for achieving mutual exclusion include using semaphores, locks and monitors.

Improper management can lead to such issues as resource starvation, where a process is constantly denied necessary resources, or priority inversion, where a lower-priority process holds a resource needed by a higher-priority process. The OS must implement strategies to handle these conflicts effectively to maintain system stability and performance.

Semaphores

Semaphores are synchronization tools used to control access to shared resources in a concurrent system. A semaphore is an integer variable that can be incremented (signal) or decremented (wait) atomically. There are two types of semaphores:

- **Binary semaphores (mutex):** Can only be 0 or 1, effectively acting as a lock to ensure mutual exclusion.

  For example: There are two processes that need to write to the same log file. To prevent both processes from writing to the file at the same time (which could cause data corruption):

  1. The semaphore is initially set to 1, indicating that the log file is available.
  2. When Process A wants to write to the log file, it checks the semaphore. If the semaphore is 1, Process A sets it to 0 (locking the resource) and proceeds to write to the log file.

3 If Process B tries to write to the log file while Process A is still writing, it will find the semaphore set to 0 and will be blocked until Process A is finished.
4 Once Process A finishes writing, it sets the semaphore back to 1, allowing Process B to proceed.
5 Process B then sets the semaphore to 0, writes to the log file, and finally sets the semaphore back to 1 when done.

- **Counting semaphores:** Can take any non-negative value, allowing multiple instances of a resource to be managed.

  For example: There is a limited number of database connections (e.g. three connections) available to a group of processes:

  1 The semaphore is initialized with a value of 3, representing the three available connections.
  2 When Process A needs a connection, it checks the semaphore. If the value is greater than 0, Process A decrements the semaphore by 1 and gains access to a connection.
  3 Process B and Process C do the same, decrementing the semaphore by 1 each time they gain access, leaving the semaphore value at 0 once all three connections are in use.
  4 If Process D then requests a connection, it finds the semaphore at 0 and must wait until one of the other processes releases a connection.
  5 When Process A finishes using its connection, it increments the semaphore by 1, signalling that a connection is now available. Process D can then proceed to use the connection.

Locks

Locks are tools used to make sure that only one process can use a shared resource at a time. For example, if two programs want to write to the same file, the first one must "lock" the file before it can start writing. If the file is already locked by another program, the second program has to wait until the lock is released. There are different types of locks, such as binary locks (also called "mutexes"), which allow only one process at a time, and readers-writer locks, which let multiple processes read a resource but only one process write to it.

Monitors

Monitors are tools used in programming to help manage access to shared resources safely. They ensure that only one process can use certain variables or methods at a time, preventing conflicts. A monitor acts like a container that holds shared variables and the code (methods) that works with them. When a process uses a monitors method, it automatically locks the monitor, so no other process can use it until the first one is done. Monitors also have condition variables that let processes wait for certain events to happen and notify others when those events occur. This makes it easier to manage and co-ordinate tasks between different processes safely.

Deadlock

Deadlock is a problem in multitasking systems where processes get stuck because each one is waiting for a resource that another process has, creating a cycle with no way to move forward. To handle deadlocks, the OS can use different strategies:

- **Deadlock prevention** involves designing the system so that deadlocks can't happen.
- **Deadlock avoidance**, such as using the Banker's algorithm, ensures a system only allocates resources if it can guarantee that all processes can eventually complete without entering an unsafe state.
- **Deadlock detection** means regularly checking for stuck processes and then taking steps to fix the problem.

- **Deadlock recovery** might involve stopping one or more processes to break the cycle or reallocating resources differently.

Multitasking challenges

The challenges of multitasking extend beyond task scheduling and resource contention. The OS must also manage context-switching efficiently, where the state of a currently running process is saved so that another process can be executed. Frequent context switches can introduce overheads, reducing overall system performance. The OS must also ensure data consistency and integrity, particularly when multiple processes access shared data. This involves implementing robust synchronization mechanisms to prevent data corruption and ensure that processes do not interfere with each other.

REVIEW QUESTIONS

1. Explain how the operating system uses task scheduling to manage multitasking. Why is it important for maintaining system performance?
2. What is resource contention, and how does the operating system resolve it to prevent issues such as resource starvation and priority inversion?
3. Describe the role of semaphores in managing resource allocation in a multitasking environment. How do binary and counting semaphores differ?
4. How does the operating system handle deadlock in multitasking systems, and what strategies can be used to prevent, avoid or resolve deadlocks?
5. What challenges does the operating system face in managing context-switching, and how does this impact overall system performance?

A1.3.6 The use of the control system components (HL)

Control systems are fundamental in automating and regulating processes across a wide range of industries, from manufacturing to robotics and environmental control. At the core of any control system are various components that work together to achieve desired outcomes by managing inputs, processing data and generating outputs. These systems rely on a precise feedback mechanism to ensure that the process remains stable and meets the set objectives.

This section explores the key elements of control systems, including the roles of the input, process, output and feedback mechanisms, as well as the critical components such as controllers, sensors, actuators and transducers, and the control algorithms that drive them. Understanding these components and their interactions is essential for designing effective and efficient control systems.

Input, process, output and feedback mechanism

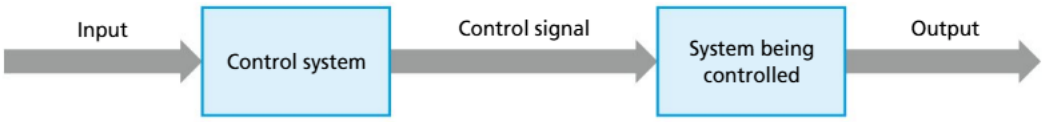
Input

In a control system, the input is the initial signal or data received by the system, representing the desired condition or target that the system aims to achieve. This input could be anything from a set temperature in a heating system to the desired speed in a motor-control application. The input is typically generated by a user, another system or an environmental condition, and serves as the reference point for the system's operation.

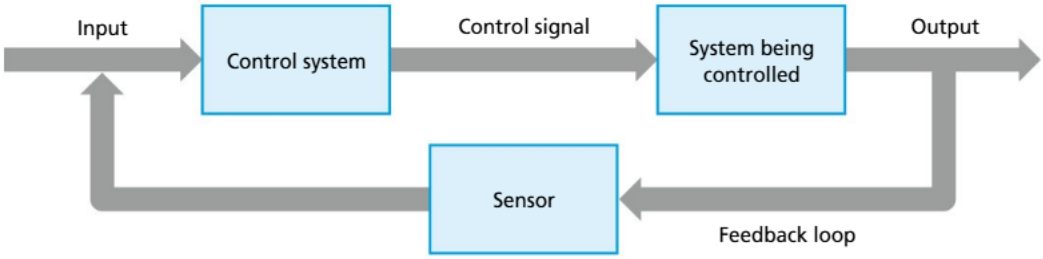
Output

The output is the result produced by the control system after processing the input. It represents the actual state or action of the system, such as turning on a heater to reach a set temperature or adjusting the speed of a motor. The output is directly influenced by the input and the control process, and it is typically the element that can be observed or measured to determine the effectiveness of the system in achieving its desired goals.

Feedback mechanism (open-loop and closed-loop)



■ Open-loop control system



■ Closed-loop control system

The feedback mechanism is a critical component in determining how a control system operates and adjusts itself to maintain desired performance.

- **Open-loop control:** In an open-loop system, there is no feedback from the output back to the input or process. The system operates solely based on the initial input without any correction or adjustment based on the actual output. This type of control is simple and used in situations where the relationship between input and output is straightforward and predictable, such as in basic timers or simple heating systems.
- **Closed-loop control:** A closed-loop system, also known as a "feedback control system", continuously monitors the output and feeds this information back into the system to adjust the process accordingly. If the output deviates from the desired input, the system makes corrections in real time to bring the output back in line with the target. This type of control is essential in applications requiring high accuracy and adaptability, such as temperature control in HVAC systems, where the system must adjust heating or cooling based on actual temperature readings.

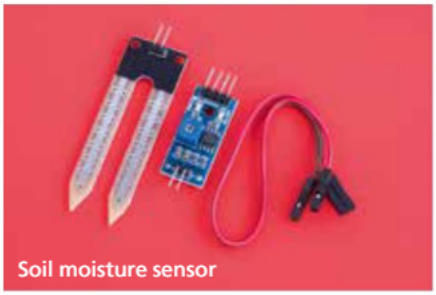
Key components

Controller

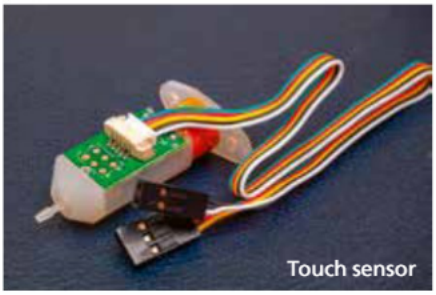
The controller is the central component of a control system that governs the operation by processing inputs and generating appropriate outputs. It acts as the "brain" of the system, implementing the control algorithm to make decisions based on the input data and feedback. The controller compares the input (desired value) with the feedback from the output (actual value) and determines the necessary actions to minimize the difference, or error, between them. This decision-making process can involve complex calculations, adjustments or commands that are sent to actuators or other system components to achieve the desired outcome. Controllers can range from simple devices such as thermostats to complex microprocessors used in industrial automation.

Sensors


Temperature and humidity sensor


Soil moisture sensor


Water / rain sensor


Touch sensor


Proximity sensor


Gas sensor

■ Different types of sensors

Sensors are devices that detect and measure physical quantities from the environment or the system itself, such as temperature, pressure, speed or light. These measurements are then converted into electrical signals that can be interpreted by the controller. Sensors serve as the "eyes and ears" of the control system, providing the necessary data for the controller to make informed decisions. The accuracy and reliability of the sensors directly impact the performance of the control system, as they provide the critical feedback needed to adjust the system's operations. For example, a temperature sensor in a climate-control system constantly monitors the room temperature, allowing the controller to adjust heating or cooling to maintain the desired setpoint.

Actuators

Actuators are the components in a control system that carry out the physical actions or adjustments in response to commands from the controller. They are responsible for converting the controller's electrical signals into mechanical motion or other forms of energy, such as turning a valve, moving a robotic arm or adjusting a motor's speed. Actuators are the "muscles" of the control system, executing the tasks that directly impact the system's output. The performance and precision of actuators are critical in applications where exact control of movements or processes is required, such as in manufacturing equipment or robotics.

Transducers

Transducers are devices that convert one form of energy into another, typically used to bridge the gap between sensors and actuators and the control system. In many cases, a sensor or actuator may not directly provide the type of signal that the controller can process or that is needed to drive the actuator. A transducer converts these signals into a compatible form. For example, a pressure sensor might detect mechanical pressure and convert it into an electrical signal that the controller can interpret. Similarly, an actuator might require a specific voltage or current that is supplied by a transducer. Transducers play a crucial role in ensuring that all parts of the control system can communicate effectively, enabling accurate and efficient operation.

Common mistake

Avoid oversimplifying the roles of control-system components.

For example, don't just say that sensors and actuators are important – explain how they work together within feedback loops to maintain system stability.

When discussing multitasking, don't just say that resource allocation is challenging – discuss specific risks, such as deadlock or priority inversion, to show a deeper understanding.

Control algorithm

The control algorithm is the set of rules or mathematical procedures that the controller uses to determine the appropriate output based on the input and feedback it receives. It is the logic that drives the decision-making process within the controller. Control algorithms can vary in complexity, from simple proportional control, where the output is adjusted in direct proportion to the error, to more advanced methods like Proportional-Integral-Derivative (PID) control, which considers past, present and future errors to make precise adjustments. The choice of control algorithm depends on the specific requirements of the system, such as the desired accuracy, speed of response and stability. A well-designed control algorithm is essential for achieving optimal performance and ensuring that the system meets its objectives efficiently and reliably.

REVIEW QUESTIONS

1. Explain the role of the controller in a control system. How does it interact with sensors and actuators to maintain system stability?
2. Describe how a feedback mechanism works in a closed-loop control system. Why is feedback essential for maintaining accuracy and stability?
3. Differentiate between open-loop and closed-loop control systems with examples. Which type is more suitable for complex, dynamic environments?
4. What is the function of a transducer in a control system? Provide an example of a transducer used in industrial automation.
5. Explain the importance of the control algorithm in a control system. How does it impact the performance and reliability of the system?

A1.3.7 Uses of control systems (HL)

Home thermostat

A home thermostat controls the room temperature by processing data from a temperature sensor that continuously monitors the environment. This sensor provides input to the thermostat, which compares the current temperature to the desired set point. If the temperature deviates from the set value, the thermostat's controller processes this information and decides whether to activate the heating or cooling system to bring the temperature back to the desired level.

The system uses a closed-loop feedback mechanism, where the temperature sensor continually feeds updated data back to the controller. As the heating or cooling system adjusts the temperature, the sensor monitors the changes and provides real-time feedback to the thermostat.

Automatic elevator control

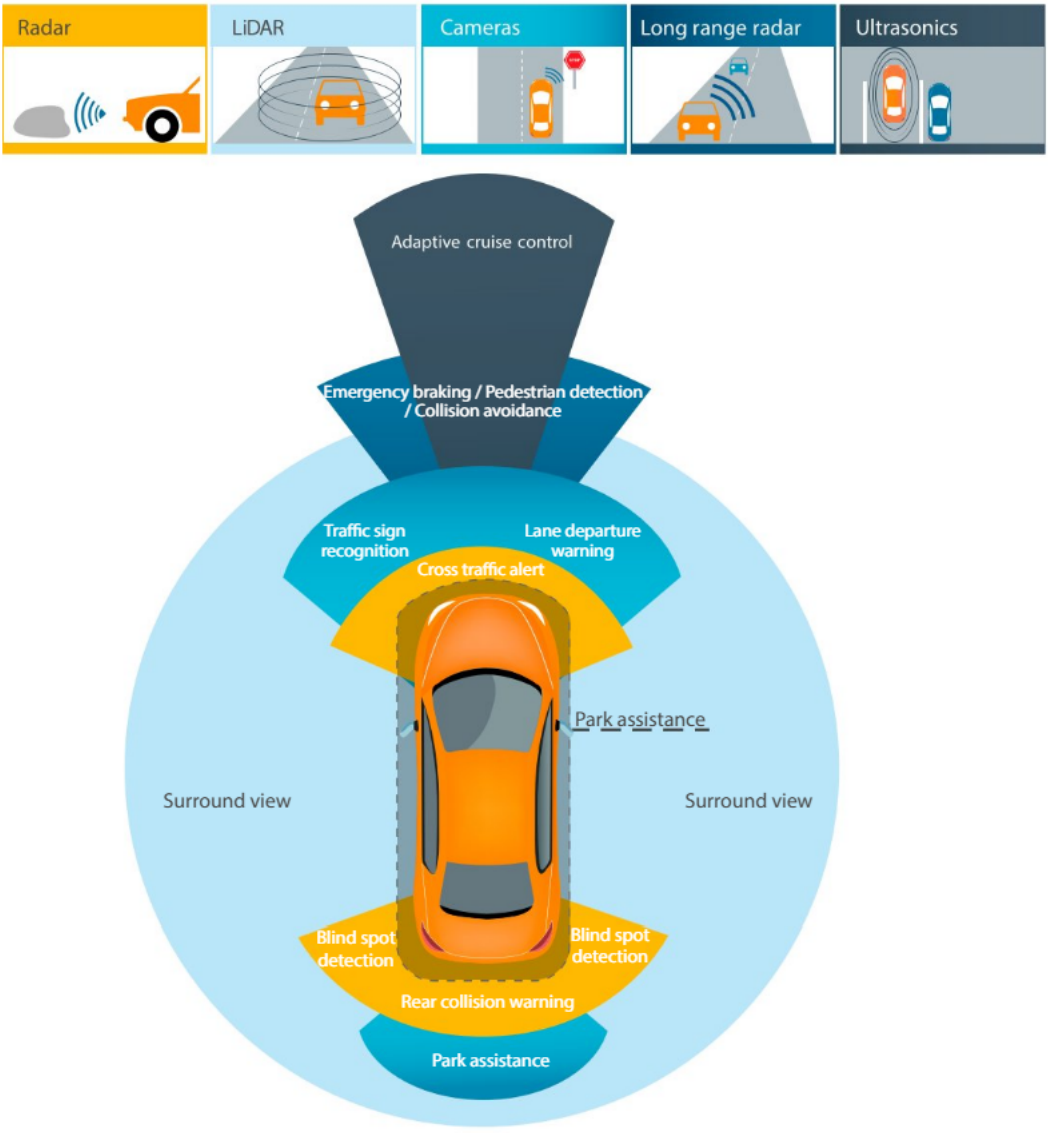
An automatic elevator control system manages the movement of an elevator by processing input from various sensors, such as those detecting the elevator's current position, floor requests and door status. These inputs are fed into the controller, which processes the data to determine the elevator's next action, such as moving up or down, stopping at a requested floor or opening and closing the doors.

The system operates using a closed-loop feedback mechanism. As the elevator moves, sensors continuously provide real-time updates to the controller about the elevator's position and speed. If the elevator needs to stop at a specific floor, the controller adjusts the motor's operation to slow down and halt the elevator precisely at the correct floor.

Autonomous vehicles

Autonomous vehicles rely on a sophisticated control system that integrates key components such as sensors, controllers, actuators and transducers to navigate and operate safely without human input. The system begins with various sensors, including cameras, LiDAR, radar and GPS, which gather critical data about the vehicle's environment, such as obstacles, road conditions and traffic signals. This data serves as the input for the vehicle's control system. The controller, often powered by advanced AI algorithms, processes this input using control algorithms to make real-time decisions about the vehicle's speed, direction and braking. The controller then sends commands to the actuators, which carry out these decisions by controlling the steering, acceleration and braking systems.

The control system operates within a closed-loop feedback mechanism, where sensors continuously monitor the vehicle's actions and environment, feeding updated data back to the controller. This allows the system to adjust its actions in real time, ensuring the vehicle can adapt to changes such as sudden obstacles or shifting traffic conditions. Transducers play a crucial role in converting sensor data into signals that the controller can process and in translating controller commands into the appropriate actions by the actuators.



The sensors used for input and output by an autonomous vehicle

Automatic washing machine

An automatic washing machine uses a control system to manage the washing process by processing input from sensors that monitor water levels, load size and cycle progress. These inputs are sent to the controller, which determines the appropriate actions, such as filling the drum with water, agitating the clothes (the motion used by the washing machine to move the clothes around in the water) or draining the water after the wash cycle.

The system operates with a closed-loop feedback mechanism, where sensors continuously update the controller on the current state of the washing process. For instance, when the water reaches the required level, the sensor signals the controller to stop filling and begin the washing cycle. Similarly, the controller adjusts the duration and intensity of the spin cycle based on the load size detected by the sensors.

Traffic signal control system

A traffic signal control system manages the flow of vehicles at intersections by processing input from sensors that detect the presence of vehicles and pedestrians and sometimes traffic conditions. These inputs are fed into the controller, which processes the data to determine the timing and sequence of the traffic lights – when to turn red, amber or green for each direction.

The system operates using a closed-loop feedback mechanism. As vehicles approach the intersection, sensors detect their presence and provide real-time updates to the controller. The controller then adjusts the signal timing based on current traffic conditions, such as extending the green light for a congested lane or triggering a pedestrian crossing light when needed. By continuously adapting to real-time data, the system optimizes traffic flow and reduces congestion, contributing to smoother and safer movement through intersections.

Irrigation control system

An irrigation control system automates the watering of agricultural fields or gardens by processing input from sensors that monitor soil moisture levels, weather conditions and sometimes the time of day. These inputs are sent to the controller, which determines when and how much water should be delivered to the plants.

The system utilizes a closed-loop feedback mechanism, where the sensors continuously update the controller on the current moisture levels in the soil. If the soil becomes too dry, the controller activates the irrigation system to deliver the appropriate amount of water. Once the desired moisture level is reached, the system shuts off the water supply. By responding dynamically to the actual needs of the soil and plants, the irrigation system conserves water and ensures optimal growing conditions, avoiding both under- and over-watering.

Home-security system

A home-security system uses a control system to monitor and protect a property by processing input from various sensors, such as door and window sensors, motion detectors and cameras. These sensors provide real-time data to the controller, which assesses potential security threats and determines the appropriate response, such as sounding an alarm, sending notifications to the homeowner or contacting emergency services.

The system operates within a closed-loop feedback mechanism, where the sensors continuously send updates to the controller about the status of the home. If a sensor detects an intrusion, the controller immediately triggers the security protocols, such as locking doors or activating cameras to record the event.

Automatic doors

An automatic-door system uses a control system to manage the opening and closing of doors by processing input from sensors that detect the presence of people or objects near the entrance. These sensors, such as infrared motion detectors or pressure mats, send signals to the controller, which then decides when to open or close the doors.

The system operates using a closed-loop feedback mechanism. As soon as the sensors detect movement or pressure, they trigger the controller to open the doors. Once the person or object has passed through and the sensors no longer detect any presence, the controller signals the doors to close.

REVIEW QUESTIONS

1. Describe how a control system operates in an autonomous vehicle. What components are involved, and how do they interact to ensure safe and efficient operation?
2. Compare the control system used in a home thermostat with that of an irrigation control system. What are the similarities and differences in how these systems maintain the desired environmental conditions?
3. Explain the importance of a closed-loop feedback mechanism in the operation of an automatic elevator control system. How does it ensure accurate and safe operation?
4. In the context of a traffic-signal control system, discuss how the control system adapts to changing traffic conditions. How does the use of sensors and feedback help in optimizing traffic flow?
5. Describe how a smart home-lighting system operates as a control system. Identify the key components, including the controller, sensors, actuators and feedback mechanism, and explain how they interact to automatically adjust the lighting in the home based on the time of day and occupancy.

PROGRAMMING EXERCISES 1

1. Simulate a traffic-light control system. If you have access to an Arduino and the components, you can build this for real. Otherwise, you can use simulation software such as Tinkercad.

   Tinkercad instructions:

   Step 1: Click on Circuits from the dashboard, and then click on Create new Circuit.

   Step 2: From the component library, search for and add the following components to your workspace:

   - ☐ One Arduino Uno R3 with breadboard
   - ☐ Three LEDs (Red, Yellow, Green) for one traffic light
   - ☐ Three resistors (220 ohms each)
   - ☐ Breadboard (optional, for better organization)
   - ☐ Pushbutton (optional, for triggering sensors)

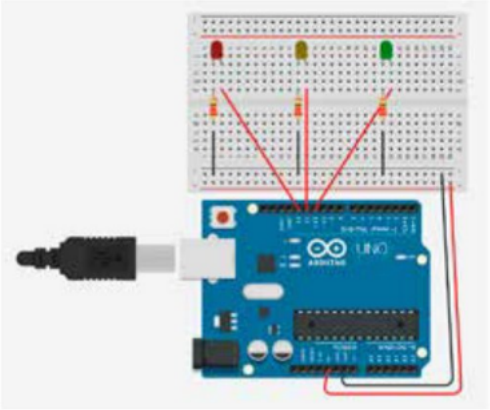
   Step 3:

   - ☐ Red LED:
     - Connect the longer leg (anode – the bent leg) of the red LED to pin 13 on the Arduino.
     - Connect the other end of the resistor to the ground (GND) on the Arduino.
   - ☐ Yellow LED:
     - Connect the anode of the yellow LED to pin 12.
     - Connect the cathode to one end of a 220-ohm resistor.
     - Connect the other end of the resistor to GND.
   - ☐ Green LED:
     - Connect the anode of the green LED to pin 11.
     - Connect the cathode to one end of a 220-ohm resistor.
     - Connect the other end of the resistor to GND.

   Step 4 (optional):

   - ☐ Place a pushbutton on the breadboard.
   - ☐ Connect one side of the pushbutton to 5V.
   - ☐ Connect the other side to a digital pin on the Arduino (e.g. pin 7).
   - ☐ Add a 10k-ohm resistor connecting the same side of the pushbutton to GND (which acts as a pull-down resistor).

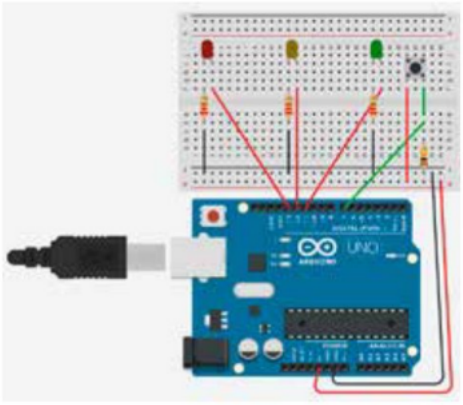
Set-up 1:



C++

```
// Pin assignments for LEDs
int redLED = 13;
int yellowLED = 12;
int greenLED = 11;
void setup() {
    // Set up the LED pins as outputs
    pinMode(redLED, OUTPUT);
    pinMode(yellowLED, OUTPUT);
    pinMode(greenLED, OUTPUT);
}
void loop() {
    // Turn on the green light for 5 seconds
    digitalWrite(greenLED, HIGH);
    delay(5000); // wait 5 seconds
    // Turn off green, turn on yellow for 2 seconds
    digitalWrite(greenLED, LOW);
    digitalWrite(yellowLED, HIGH);
    delay(2000); // wait 2 seconds
    // Turn off yellow, turn on red for 5 seconds
    digitalWrite(yellowLED, LOW);
    digitalWrite(redLED, HIGH);
    delay(5000); // wait 5 seconds
    // Turn off red, and repeat the cycle
    digitalWrite(redLED, LOW);
}
```

Set-up 2 (with optional pushbutton):



C++

```
int buttonPin = 7; // Pin for the sensor (pushbutton)
int buttonState = 0;
int greenLED = 11; // Pin for green light
int yellowLED = 12; // Pin for yellow light
int redLED = 13; // Pin for red light
unsigned long previousMillis = 0; // Variable to store the last time the light
// changed
const long greenInterval = 5000; // Duration the green light stays on
// (in milliseconds)
const long yellowInterval = 2000; // Duration the yellow light stays on
// (in milliseconds)
const long redInterval = 5000; // Duration the red light stays on
// (in milliseconds)
// Define possible states for the traffic light
enum LightState {GREEN, YELLOW, RED};
LightState currentState = GREEN; // Start with the green light on
void setup() {
    pinMode(buttonPin, INPUT); // Set the pushbutton pin as an input
    pinMode(greenLED, OUTPUT); // Set the green LED pin as an output
    pinMode(yellowLED, OUTPUT); // Set the yellow LED pin as an output
    pinMode(redLED, OUTPUT); // Set the red LED pin as an output
    digitalWrite(greenLED, HIGH); // Initially turn on the green light
}
void loop() {
    unsigned long currentMillis = millis(); // Get the current time in milliseconds
    buttonState = digitalRead(buttonPin); // Read the state of the pushbutton
    switch(currentState) {
        case GREEN:
            // If the button is pressed or the green light has been on for the
            // full interval
            if (buttonState == HIGH || currentMillis - previousMillis >=
            greenInterval) {
```

```
                digitalWrite(greenLED, LOW); // Turn off the green light
                digitalWrite(yellowLED, HIGH); // Turn on the yellow light
                currentState = YELLOW; // Change the state to YELLOW
                previousMillis = currentMillis; // Reset the timer to the
                // current time
            }
            break;
        // The YELLOW and RED cases follow similar logic
        case YELLOW:
            if (currentMillis - previousMillis >= yellowInterval) {
                digitalWrite(yellowLED, LOW);
                digitalWrite(redLED, HIGH);
                currentState = RED;
                previousMillis = currentMillis; // Reset the timer to the
                // current time
            }
            break;
        case RED:
            if (currentMillis - previousMillis >= redInterval) {
                digitalWrite(redLED, LOW);
                digitalWrite(greenLED, HIGH);
                currentState = GREEN;
                previousMillis = currentMillis; // Reset the timer to the
                // current time
            }
            break;
    }
}
```

2 Observe how the control system manages the traffic-light sequence and adapts to changes (if a sensor is used).

3 Discuss how the components (LEDs, controller, optional sensor) work together as part of the control system.

4 Discuss the effectiveness of your traffic-light control system and how it could be improved or extended.

PROGRAMMING EXERCISES 2

1 Simulate a motor control system where the speed of the motor is adjusted based on feedback from a sensor. It also has a maximum speed that is set in code and should not be exceeded by the motor. If you have access to an Arduino and the components, you can build this for real. Otherwise, you can use simulation software such as Tinkercad.

Tinkercad instructions:

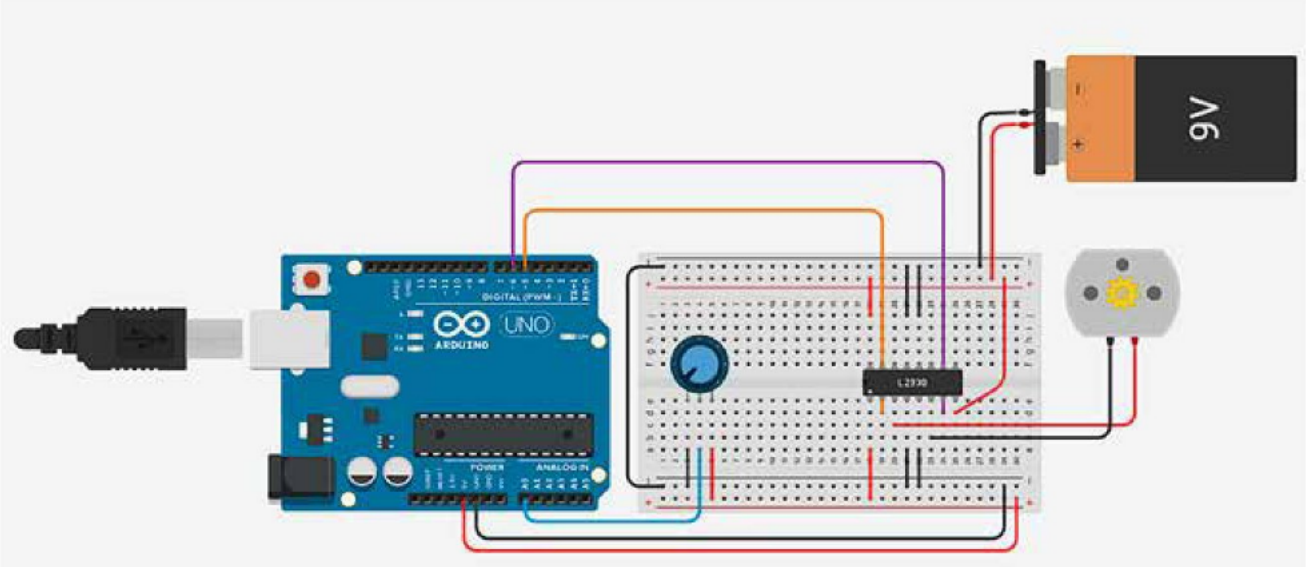
Step 1: Click on Circuits from the dashboard, and then click on Create new Circuit.

Step 2: From the component library, search for and add the following components to your workspace:

- ☐ One Arduino Uno R3 with breadboard
- ☐ One potentiometer
- ☐ H-Bridge motor driver (L293D)
- ☐ One DC motor
- ☐ One 9V battery

Step 3:

- ☐ Arduino:
  - Connect 5V to the bottom positive (red) power rail on the breadboard.
  - Connect GND to the bottom negative (black) ground rail on the breadboard.
  - Connect the bottom negative (black) ground rail on the breadboard to the top negative (black) ground rail on the breadboard.
- ☐ Potentiometer:
  - Connect Terminal 1 to the ground (GND) on the Arduino.
  - Connect Terminal 2 to the 5V on the Arduino.
  - Connect Wiper to A0.
- ☐ 9V battery:
  - Connect the positive terminal of the battery to the top positive (red) power rail on the breadboard.
  - Connect the negative terminal of the battery to the top negative (black) ground rail on the breadboard.
- ☐ H-Bridge motor driver (L293D):
  - Place it, ensuring that it straddles the centre gap between the two sides of the breadboard.
  - Connect "Enable 1 & 2" to the 5V on the Arduino.
  - Connect Power 1 to the 9V on the battery.
  - Connect all four ground pins to the GND.
  - Connect Power 2 to the top positive (red) rail on the breadboard.
  - Connect Input 1 to pin 5 on the Arduino.
  - Connect Input 2 to pin 6 on the Arduino.
- ☐ DC motor:
  - Connect Terminal 1 to Output 2 on the L293D.
  - Connect Terminal 2 to Output 1 on the L293D.



C++

```
int potValue; // Variable to store the potentiometer input value
int maximumSpeed = 128; // Maximum motor speed (should not be exceeded)
int forwardPin = 5; // Pin connected to the forward control input on the
// motor driver
int reversePin = 6; // Pin connected to the reverse control input on the
// motor driver
```

```
void setup() {
    pinMode(forwardPin, OUTPUT); // Set the forward pin as an output
    pinMode(reversePin, OUTPUT); // Set the reverse pin as an output
    Serial.begin(9600); // Initialize serial communication at 9600 baud for
    // debugging
}
void loop() {
    potValue = analogRead(A0); // Read the analog value from the potentiometer
    // (0-1023)
    int motorSpeed = map(potValue, 0, 1023, 0, 255); // Scale the potentiometer
    // value to match PWM range (0-255)
    // Ensure the motor speed does not exceed the desired value
    if (motorSpeed > maximumSpeed) {
        motorSpeed = maximumSpeed;
    }
    // Write the PWM value to the forward pin
    analogWrite(forwardPin, motorSpeed);
    analogWrite(reversePin, 0); // Ensure reverse pin is off
    // Print the motor speed value and desired speed to the serial monitor
    Serial.print("Motor Speed: ");
    Serial.print(motorSpeed);
    Serial.print(" | Desired Speed: ");
    Serial.println(maximumSpeed);
    delay(100); // Short delay to make the serial output readable
}
```

2 Observe how the motor speed changes as the sensor value changes. For example, turning the potentiometer should increase or decrease the motor speed, depending on the direction.

3 Modify the `maximumSpeed` in the code to see how the system responds to different target speeds or positions.

EXAM PRACTICE QUESTIONS

1 Describe the role of an operating system in organizing files within a directory structure. [2]

2 Outline the steps an operating system takes to load an application into memory. [3]

3 Describe the function of memory management within an operating system. [3]

4 Describe the function of process scheduling in an operating system. [3]

5 Describe how an operating system ensures security through user authentication. [2]

6 Compare the advantages and disadvantages of using first come first served (FCFS) and round robin (RR) scheduling algorithms in operating systems. [4]

7 Explain how priority scheduling can cause some processes to be ignored or delayed in an operating system. Describe a way to prevent this problem. [4]

8 Discuss how event frequency and CPU processing overheads influence the choice between interrupt handling and polling. [4]

9 Describe the role of the operating system in preventing deadlock during multitasking. [3]

10 Describe how an irrigation control system operates as a control system. What components are involved, and how do they interact to ensure optimal watering conditions? [4]